

Introduction Kotlin

val immutable - assignable une fois	var mutable
val p : Person?; p?.name	Null safety: Pour chaque type T, il existe T? qui prend T ou null
c?.name ?: "default"	Elvis Operator: Prendre valeur en cas de null
fun String.doSomething():String {...}	Extension functions: ajouter fonctions à classes sans sources
class Person(var i:Int) {init {...}}	init: constructeur par défaut
constructor(i:Int,j:Int):this(i) {...}	constructor: constructeurs supplémentaire
fun add(x: Int, y: Int):Int {...}	Arguments nommés possible: add(y = 1, x = 2)
class Student(...): Person(...) {...}	Héritage: appel super-constructeur. open class Person() {...}
interface Flying { val w:Wings fun fly(){...} }	class Bird: Flying { override val wings:Wings=... }
data class Person(var name:String)	Génère: getter - setters - toString()
val(name, surname) = p	Déstructuration pour data class
var p2 = p1.copy(surname="Bob")	Copie pour data class, arguments optionnels
val s = "Person[name:\${p.name},id:\${id}]"	String templates
Collections	List, MutableList, Set, MutableSet, Map, MutableMap
val l = listOf(1, 2, 3, 4)	l.any{...} l.count{...} l.max() l.filter{...} l.map{...} l.partition{...} l+l etc
Operators overload:	Array: get(i) set(i) a..b b.rangeTo(b) a in b b.contains(a)
Unary: unaryPlus() not() inc() dec()	Binary: plus(b) minus(b) times(b) div(b) mod(b) plusAssign()
val res = when(x) {1->"a" else->{"b"}}	When - Pattern Matching
p?.let {it.name="Bob" println(it)}	Scope functions - let - retourne dernière ligne du bloc
val p2 = p.apply {this.name="Bob" println(this)}	Scope functions - apply - retourne l'objet lui-même
BufferedWriter(FileWriter("a.txt")).use{ it.appendLine("Hello") }	Scope functions - use - appelé sur objets <i>Closeable</i> , ferme la variable it automatiquement en fin de bloc ou en cas d'exception
companion object {var a fun getA():A}	Companion objects contient variables/fonctions statiques

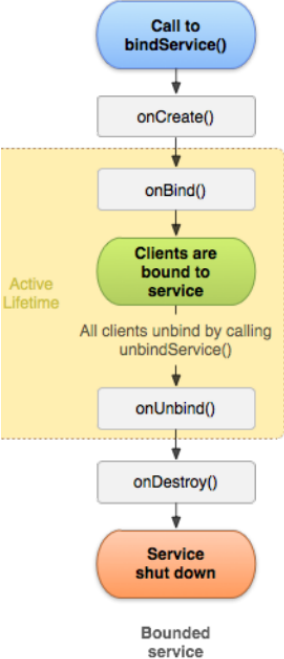
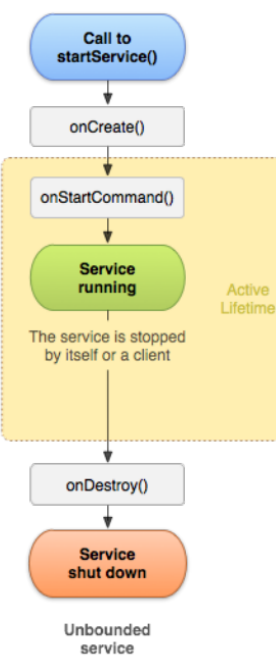
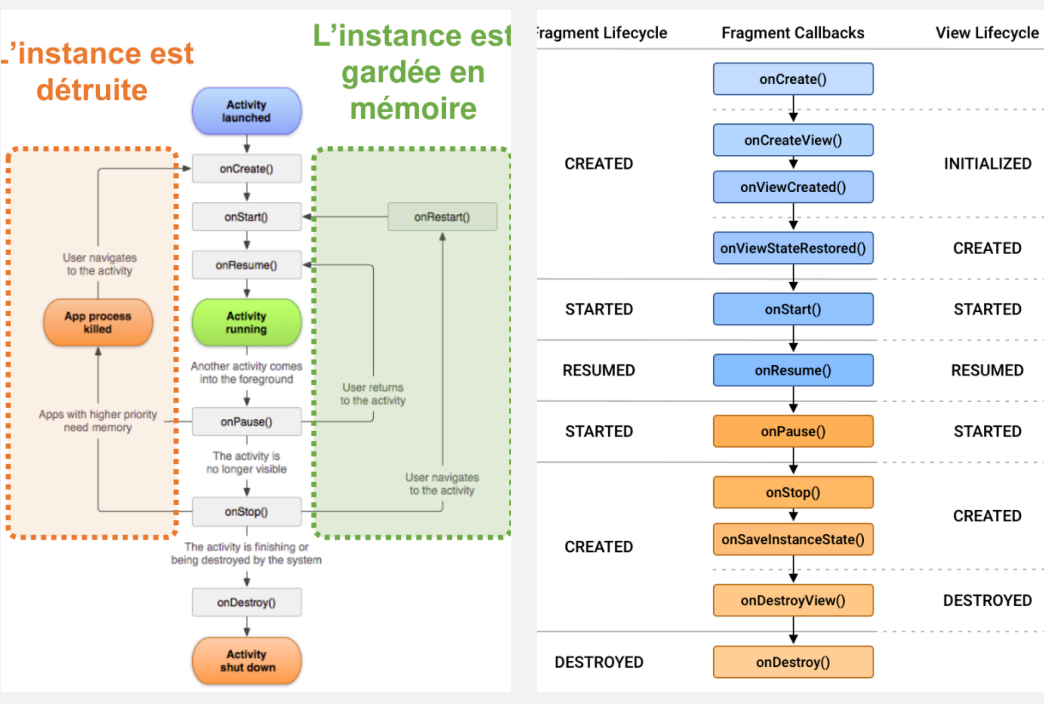
Ressources

Manifest	décrit informations essentielles pour build, OS, et store
Doit contenir	Composants d'app(Activités, Services, Broadcast receivers, Content providers), Permissions, Fonctionnalités (hard/software)
Ressources	différentes ressources et classe <i>R</i>
Contextualisation - ⚠ Lire doc pour ordre précis	Plusieurs ressources pour différents contextes/configurations. chargées automatiquement lors exécution la mieux adaptée. ex : <i>values-fr sw600dp night land</i> densité écran: <i>hdpi</i> taille écran: <i>normal</i>
Valeurs - string.xml	peut être regroupées en tableau
Placeholders	<string name="welcome">Hello, %1\$s!</string>

Plurals - zero one two few many	<plurals name="test"> <item quantity="one">one</item> <item quantity="other">more</item> </plurals>
Dimensions - dimension.xml	dp:density independent pixel, sp:scale independent pixel
Couleurs - colors.xml	-> themes.xml
Drawables	
Bitmap - png, webp, jpeg, gif	Plusieurs résolutions pour chaque type d'écran. résolutions différentes via contexte de densité.
Vector - xml	outil pour convertir depuis svg ou psd
Nine-Patch - *.9.[png]	Redimensionnement contrôlé
State List	Drawable matchant différents states de la vue
Level List	Drawable matchant une valeur numérique
Layouts - LinearLayout - RelativeLayout - ConstraintLayout - ScrollView	possible de les imbriquer mais moins bonnes performances. ScrollView est vertical (mais HorizontalScrollView existe)
classe R	regroupe les ids uniques des ressources. Accessible code ou ressources, autogénérée lors du build dans dossier <i>gen</i> à la racine du package. setContentView(R.Layout.activity_main)
Build - Gradle	build.gradle app et projet
Dépendances - Maven	via <i>build.gradle.kts</i> ou <i>libs.versions.toml</i> (mieux)

Activités, Fragments, Services	
Activity - Stack	Doivent être déclarées dans le manifest
Cycle de vie - Inact->Stopped->Paused->Active->Paused->Stopped->Inact	Active:foreground, can not be killed Paused:visible, no focus Stopped:invisible Inactive:temporary when created/killed
viewBinding	option buildFeatures dans build.gradle app, automatise linkage des vues, génère une classe pour chaque layout
Cycles de vie	application réagit aux changement d'états du lifecycle via méthodes de callback (onCreate, onStart, ...)
Initialisation	initialiser vue via viewBinding ou classe R et findViewById
Nouvelle activité	nouvelle lancée via Intent, précédente dans la pile en <i>Stopped</i>
Navigation entre activités	configurations système peuvent entrainer recréation activité, si celle-ci supprimée car plus de ressource entre temps également -> sauver état courant dans bundle
Terminer activité courante	Back -> onPause -> onStop -> onDestroy -> finish
État activité	activités sur stack peuvent être détruites. changement de config va recréer activité active
Sauvegarde-Restoration	onSaveInstanceState -> Bundle -> onCreate/ onRestoreInstanceState
Intents	Lance une activité sur la stack avec paramètres éventuels
Intents Explicites - activité précise	Intents Implicites - appel générique via un lien/type intent
Contracts	résultat d'une activité (moderne, non déprécié)
Fragment	plusieurs par Activity. Gérés par Fragment Manager
Service	Réaliser longues opérations en arrière-plan
Foreground - par intent ou méthode	notification visible: Lecteur Audio, Téléchargement,...
Background - par intent ou méthode	sans interface utilisateur, limité dans le temps: sync. serveur
Bounded - par méthode, bind	lié composant d'une app, détruit lorsque plus aucun n'est bind.
Broadcast Receivers	publish-subscribe, via manifest, runtime (moins de limitations)
Content Providers	accéder DB, Uri unique par donnée, CRUD
FileProvider - sous classe	partage sécurisé de fichiers entre apps à private storage
Permissions - Bonnes pratiques	Contrôle, Transparence, Minimisation
Installation	listées dans manifest, automatiquement accordées
Exécution	permissions dangereuses, listées dans manifest, pop up
Spéciales	réservées à OS ou constructeur téléphone

Diagrams



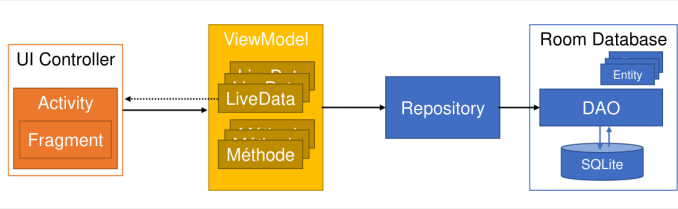
Interface graphique	
UI-Thread - ex. fréquence 16ms	affichage graphique, ne doit pas être surchargé car usage fréquent
Autres threads	Communications(HTTP,Socket), DB, Périphériques(Bluetooth,NFC)
Material Components	librairie Google de widgets respectant ligne graphique
Types de vue	
Visibilité	VISIBLE, INVISIBLE (prend de la place), GONE (n'en prend pas)
<i>TextView</i>	afficher du texte à l'utilisateur
<i>EditText</i>	saisie de texte par l'utilisateur, spécifier type de valeur
<i>TextField</i>	meilleur que EditText. propriétés erreur, icone, ...
<i>Button</i>	interagir avec utilisateur via clic
<i>ImageView</i>	afficher une image depuis ressources ou mémoire
<i>ImageButton</i>	<i>Button</i> et <i>ImageView</i>
Choix binaires	<i>CheckBox</i> , <i>Switch</i> (simple bouton on-off), <i>ToggleButton</i> (avec texte)
<i>RadioGroup</i>	qui contient des <i>RadioButton</i>
<i>Spinner</i>	liste de choix dans un menu déroulant
<i>ProgressBar</i>	barre de chargement animée. indéterminé ou non
<i>SeekBar</i>	renseigner un avancement (lecture youtube/musique)
<i>WebView</i>	afficher contenu web. basé sur chrome. requière permission Internet. utiliser pour charger contenu local. Js peut appeler code app et vice-versa
Feedback	
<i>Toast</i>	pop up furtif, en bas de l'écran
<i>SnackBar</i>	pop up en bas d'écran, possible d'ajouter action
<i>Dialog</i>	pop up demandant de prendre une décision: <i>AlertDialog</i> , <i>DatePickerDialog</i> , <i>ProgressDialog</i> , <i>FragmentDialog</i>
Notifications	
	message asynchrone affiché en dehors de l'app. actions possibles. Associé à un canal lors de sa création (identifiant unique)
Expendable	permet d'aggrandir la notification pour afficher plus de texte
ActionBar	"menu", affiche par défaut titre app, peut accueillir actions, icones,...
ListView	affichage vertical défilant d'une collection d'éléments
RecyclerView	impose recyclage vues, objets différents types et layouts, animations intégrées ajout/update/delete éléments, scroll vertical/horizontal, DiffUtils, + performant que ListView, + complexe, manque fonctionnalités (OnItemClickListener)
DiffUtils	ne rafraichit que les éléments modifiés
Autres widgets	
<i>Floating Action Button</i> - FAB	bouton flottant au dessus de interface, en bas à droite,.
<i>GestureDetector</i>	détecte les actions utilisateurs habituelles avec plusieurs doigts

LiveData et MVVM	
LiveData - librairie Android Jetpack	observable, lifecycle aware (ne va notifier que si actif/visible)
avantages:	update visuelle auto, pas de memory leaks
souvent créés dans un ViewModel	remplace sauvegarde état, partage données entre activité/fragment
MutableLiveData	LiveData ne sont pas modifiables
MVC - Responsabilités Activités/ Fragments	gestion vues (init, update), réaction user input, API system calls, ...
MVC - Problèmes Activités/ Fragments	destruction/restoration à tout moment, perte d'appels asynchrones
MVVM	Model (Room) <-> ViewModel <-> View (UI controller)
ViewModel	si associé à une activité survit aux recreations (rotation), référencé Activités/ Fragments, stockage données court terme/sans persistance
particularités	n'est instancié que si utilisé, si utilisé lorsqu'activité inactive->échec
bonnes pratiques	ne pas avoir de réf vers Vue/Activité/Fragment, plusieurs par Activité, exposer LiveData uniquement et pas MutableLiveData (mais possible de cast en MutableLiveData...)

Persistance des données

dossier <i>assets</i>	disponible lecture seule
Fichiers privés - interne: chiffré	géré par application - mise en cache: sans garantie
Fichiers privés - externe: chemin absolu peut changer	géré par application - mise en cache
Fichiers média partagés	API <i>Mediastore</i> requêtes et résultats en Uri
SharedPreferences	key-value, géré par SDK, stockés dans XML privé interne
Base de données	Données stockées dans un fichier, SQLite
Android Room	ORM pour SQLite
KSP - Kotlin Symbol Processing API	génération de code par annotation
Repository	Single source of truth, point d'accès aux données
Entités	data class pour stocker objets, annotations
DAO	Data Access Object, interfaces pour accéder à la DB, requêtes, code généré automatiquement
Database	Définition DB, mise en relation composants, singleton
Converters	convertit types complexes afin de les stocker, par ex date
Requêtes sur LiveData	Exécuter chaque fois que table est modifiée -> <i>distinctUntilChanged</i>
Requêtes sur toutes les données	Charger en mémoire -> <i>Flow</i> ou pagination
non chiffrement des données	DB: Android 10 pour stockage interne. applicatif: SQLite+SQLCipher

Architecture



Persistance des données - Résumé

Type	Permissions	Accès autres applications	Suppression lors désinstallation
Fichiers privés: Internes	aucune	non	oui
Fichiers privés: Externes	aucune (19+)	possible	oui
Média - <i>Mediastore API</i>	READ_EXTERNAL_STORAGE	oui	non
Autres fichiers partagés (téléchargements)	aucune via <i>Storage Access Framework</i>	oui	non
Préférences	aucune	non	oui
Base de données locale	aucune	non	non

Threads, coroutines	
Threads	Main Thread (UI), Communications, DB, Périphériques
Handler	Associé à un thread, permet de revenir dans le UI thread
Limites d'utilisation	bien pour tâches courtes, ponctuelles. persistent si activité détruite. pas garbage collecté d'activity si réf de thread vers activity. concurrence entre threads. pas conscience cycle de vie Android.
Coroutines	Appel séquentiel de différentes opérations.
Suspending function	Profite du partage de thread.
Blocking function	opérations IO, longs calculs, ...
Dispatchers	Main : UI Thread, unique thread, Default : CPU, nb de threads = nb de CPU, IO : méthodes bloquantes, nb de threads = dynamique, max 64
GlobalScope	Scope application, au développeur de les stopper, déconseillé
LifeCycleScope	associé à l'objet avec cycle de vie (Activity ou Fragment), automatiquement stoppés
ViewModelScope	comme LifeCycleScope mais pour ViewModel
WorkManager	Immediate, Long Running (+10min), Deferrable(programmed/periodic)
périodique	Android Doze, App Standby Buckets, App hibernation
Android Doze	si verrouillé sans chargeur -> veille profonde interrompue par tâches lors d'une maintenance window
App Standby Buckets	classer apps selon utilisation : Active, Working set (quotidienne), Frequent (hebdomadaire), Rare (sporadique), Restricted (pas ouvert depuis +8 jours)
App hibernation	si pas d'interactions pendant "longtemps" -> permissions runtime révoquées, plus de tâches programmées, plus de messages push, dossiers cache vidés

Communications

Wi-fi	connection ponctuelle
Réseaux mobiles	2-5G, accès continu dans un pays, intercellulaire
Accès Internet	via permission dans manifest. possible de spécifier fichier de config pour options
Services web	RESTful : GET POST PUT/PATCH DELETE
java.net.URL	pour interroger API REST. GET par défaut.
gson	librairie pour désérialiser json <code>GSON().fromJson<Object>(json, type)</code>
alternatives à java.net.URL	OkHTTP, Volley, Retrofit

Synchronisation des données	DB locale synchronisée avec serveur lorsque internet de retour
DB locale	possède champ "status" : ok, new (depuis local), mod, del. possède id local et remote_id (null si new)
Limites	Pas valable si d'autres mobiles utilisent la même DB. Repository de l'app = single source of truth

Jetpack Compose

Bases	Interface déclarative. Kotlin uniquement. simplifier/accélérer conception UI. généré par code.
-------	--

ensemble de libraires	Gradle propose Bill of Materials (BOM) qui gère quelle version est compatible avec quoi
How To	@Composable : Bloc de construction UI -> <code>setContent{MyComposeApplicationTheme{Hello()}}</code>
Fonctions composable	appelées 60/s, pas recomposer ce qui ne change pas, composition multi threading -> doit être idempotent, rapide, pas d'effet de bords
Layouts	Column, Row (LinearLayout), Box (RelativeLayout)
Sucre	si le dernier paramètre d'une fonction est une fonction, la fonction lambda peut être placé hors parenthèses. si pas d'autres, les parenthèses peuvent être omises
ConstraintLayout	disponible via librairie, il faut référencer les fonctions dans celui ci via des identifiants
Scaffold	permet de définir où les éléments de l'interface seront placés
Fonctions paresseuses	ListView/RecyclerView -> Lazycolumn/LazyRow/LazyVerticalGrid. pas de recyclage, systématiquemen recomposées
Gestion des états	cycle de vie : première composition, éventuelles recompositions, quitte la composition
remember	stocke un unique objet d'état en mémoire (Stable<T> ou MutableStable<T>) créé et initialisé à première composition. toute modification va recomposer la vue. lorsque composition plus visible, objet est oublié.
rememberSaveable	automatise sauvegarde/restauration dans le Bundle
State Hoisting	on veut sortir l'état du composant -> value, onValueChange -> Single source of truth, encapsulation, interceptable, partage, découplage

form factor	éléments racines qui occupent toute la place, composables traditionnels qui sont contenus dans racines
réutilisables	fonctions non racines doivent pouvoir l'être. Doit se baser sur espace donné.
Divers	pas recommandé de mélanger layout xml et compose, mais possible !

Tests

Catégories	tests unitaires (fonctionnalités) - tests d'intégration (modules) - tests de bout en bout (workflow)
Particularités Android	development et build par réalisés sur cible. SDK ne fournit pas implémentation de la plupart des classes/fonctions. bcp d'asynchronisme. interface graphique = animations, transitions
Tests Unitaires	placés dans <code>app/src/test/java/</code> utilisation de <code>JUnit @Test</code> contexte : <code>@get:Rule</code>
Instrumentalisé Avec Room	<code>@LargeTest @Before setUp() @After tearDown() @Test</code>
Instrumentalisé Avec LiveData	<code>dao.count().awaitingValue()</code> pour attendre valeur. Limitation jusqu'à NBR_ITEM_TO_KEEP (100)
Instrumentalisé Avec GUI	désactiver animations graphiques,
Particularités	si trop de travail Thread-UI -> ANR -> pas interface prévue. activité peut prendre + de temps à démarrer -> erreurs
Instrumentalisé Avec Compose	Tester chaînes de caractères -> <code>!</code> refactoring, possible d'ajouter identifiants aux fonctions composables
CI/CD	SDK Android requis, tests instrumentalisés via émulateur, setup via cli puis adb

Playstore - Monkey testing	testée automatiquement lors de publication. données aléatoires entrées. possible de renseigner un identifiant, sur plusieurs appareils. vérification accessibilité, failles de sécurité.
Firebase - Robo Tests	Monkey testing basé sur Tests Robo du Test Lab de Firebase. API disponible, payant pour large échelle. Cartographie de l'app, captures d'écran, logs, profilage, etc...